

MODULE 3

Transaction Management and Concurrency Control

What is a Transaction?

- Collections of operations that form a single logical unit of work are called **transactions**.
- A database system must ensure proper execution of transactions despite failures—either the entire transaction executes, or none of it does.
- **Transaction** is a unit of program execution that accesses and possibly updates various data items.

In database terms,

- ❖ A transaction is any action that reads from or writes to a database. A transaction may consist of the following:
 - A simple SELECT statement to generate a list of table contents.
 - A series of related UPDATE statements to change the values of attributes in various tables.
 - A series of INSERT statements to add rows to one or more tables.
 - A combination of SELECT, UPDATE, and INSERT statements.
- ❖ Example: The sales transaction example includes a combination of INSERT and UPDATE statements.

- A *logical* unit of work that must be either entirely completed or aborted
- Successful transaction changes the database from one *consistent* state to another
 - One in which all data integrity constraints are satisfied
- Most real-world database transactions are formed by two or more database requests
 - The equivalent of a single SQL statement in an application program or transaction

Evaluating Transaction Results

Evaluating Transaction Results

- Not all transactions update the database. Suppose that you want to examine the CUSTOMER table to determine the current balance for customer number 10016. Such a transaction can be completed by using the following SQL code:
- `SELECT CUST_NUMBER, CUST_BALANCE FROM CUSTOMER WHERE CUST_NUMBER = 10016;`
- Although the query does not make any changes in the CUSTOMER table, the SQL code represents a transaction because it accesses the database.
- If the database existed in a consistent state before the access, the database remains in a consistent state after the access because the transaction did not alter the database.

TRANSACTION CONCEPT

- A transaction is a **unit of program execution** that accesses and possibly updates various data items.
- A transaction is made to change data in a database which can be done by inserting new data, updating the existing data, or by deleting the data that is no longer required.
- There are certain types of transaction states which tell the user about the current condition of that database transaction and what further steps to be followed for the processing.
- **Read(X)**: This read operation is applied to read the X's value from the database server and keeps it in a buffer in the main memory.
- **Write(X)**: This write operation is applied to write the X's value back to the database server from the buffer.

Example: transaction to transfer \$50 from account A to account B:

1. read(A)
2. $A := A - 50$
3. write(A)
4. read(B)
5. $B := B + 50$
6. write(B)



➤ Two main issues to deal with:

- Failures of various kinds, such as hardware failures and system crashes - **Recovery**
- **Concurrent execution** of multiple transactions

Transaction Properties(ACID properties)

About ACID Properties



- It is important to ensure that the database remains consistent before and after the transaction.
- To ensure the consistency of database, certain properties are followed by all the transactions occurring in the system.
- These properties are called as ACID Properties of a transaction.

Atomicity

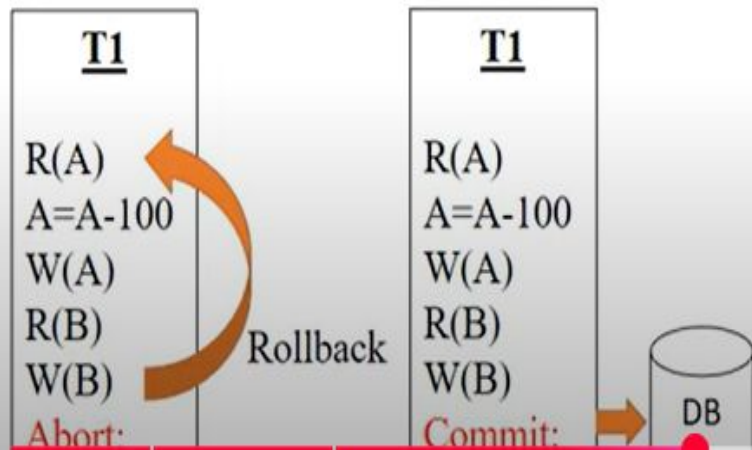
- Requires that **all** operations (SQL requests) of a transaction be completed; if not, then the transaction is aborted
- A transaction is treated as a single, indivisible, logical unit of work
- This “**all-or-none**” property is referred to as atomicity.
- Example: If a transaction T1 has four SQL requests, all four requests must be successfully completed; otherwise, the entire transaction is aborted.
- In other words, a transaction is treated as a single, indivisible, logical unit of work.

1. Atomicity

➤ (ALL or NONE)

- This property ensures that either the transaction occurs completely or it does not occur at all.
- In other words, it ensures that no transaction occurs partially.
- It is the responsibility of Transaction Control Manager to ensure atomicity of the transactions.
- The operations either get Abort or Commit.

Examples:



The screenshot displays the "Movie Ticket Booking" application interface, which is divided into three main sections:

- User Authentication:** Includes fields for "User name" and "Password", a "Login" button, and a "Register a new User" button.
- Selection Options:** Features dropdown menus for "Select Center", "Select Movie", "Select Date", and "Select Time", along with "Book Now" and "Reset" buttons.
- Ticket Selection:** A section titled "PLEASE SELECT CLASS AND NO. OF TICKETS" with input fields for "Class", "TICKET PRICE", "COMBO PRICE", "SERVICE FEES", and "NET PAYABLE", and a "Proceed To Pay" button.

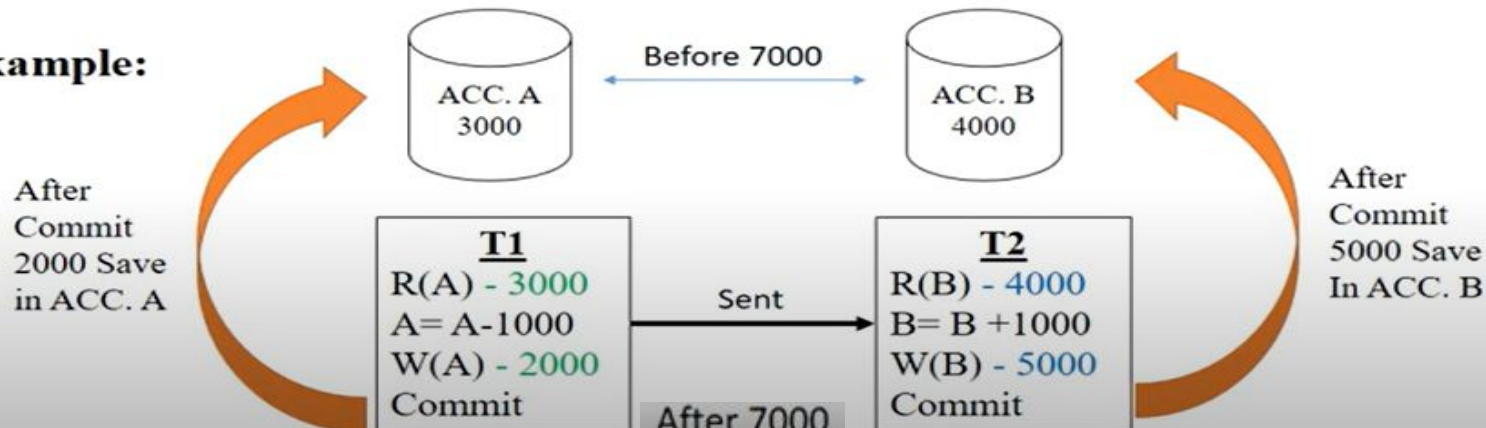
Consistency

- Consistency property ensures that the database must remain in **the consistent state before the start of transaction and after the transaction is over.**
- Consistency states that only valid data will be written to the database.
- If for some reason a transaction is executed that violates the database consistency rules the entire transaction will be rolled back.

➤ (Before & After the Transaction, Sum same)

- This property ensures that integrity constraints are maintained.
- It ensures that the database remains consistent before and after the transaction.
- It is the responsibility of DBMS application programmer to ensure consistency of the database.

Example:



Isolation(This occurs before commit statement)

- Data used during execution of a transaction cannot be used by second transaction until first one is completed
- Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
- In other words, if transaction
- T_1 is being executed and is using the data item X , that data item cannot be accessed by any other transaction ($T_2 \dots T_n$) until T_1 ends. This property is particularly useful in multiuser database environments because several users can access and update the database at the same time.

- **Durability**

Durability ensures that once transaction changes are done and committed, they cannot be undone or lost, even in the event of a system failure.

- After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.
- Durability can be implemented by writing all transaction into a **transaction log** that can be used to create a system state right before failure.
- A transaction can only regard as committed after it is written safely in the log.
- For example, in an application that transfers funds from one account to another, the durability property ensures that the changes made to each account will not be reversed.
- These properties are called the **ACID properties**.

Transaction State

A transaction must be in one of the following states:

- **Active:-**

- The initial state; the transaction stays in this state while it is executing.

- **Partially committed:-**

- After the final statement has been executed.

- **Failed:-**

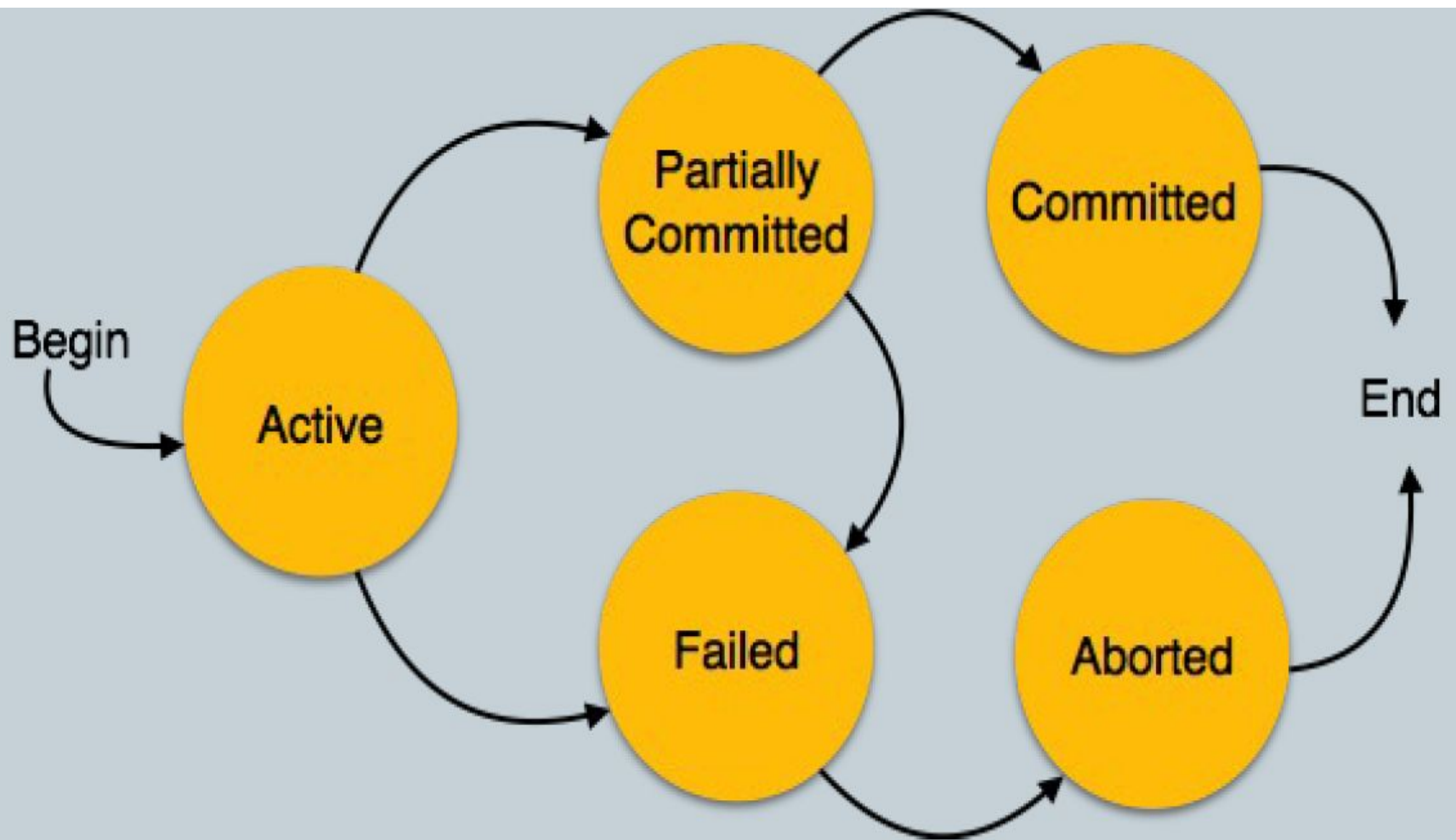
- After the discovery that normal execution can no longer proceed.

- **Aborted:-**

- After the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction

- **Committed:-**

- After successful completion



Transaction Management with SQL

The American National Standards Institute (ANSI) has defined standards that govern SQL database transactions. Transaction support is provided by two SQL statements: COMMIT and ROLLBACK. The ANSI standards require that when a transaction sequence is initiated by a user or an application program, the sequence must continue through all succeeding SQL statements until one of the following four events occurs:

- A COMMIT statement is reached, in which case all changes are permanently recorded within the database. The COMMIT statement automatically ends the SQL transaction.
- A ROLLBACK statement is reached, in which case all changes are aborted and the database is rolled back to its previous consistent state.
- The end of a program is successfully reached, in which case all changes are permanently recorded within the database. This action is equivalent to COMMIT.
- The program is abnormally terminated, in which case the database changes are aborted and the database is rolled back to its previous consistent state. This action is equivalent to ROLLBACK.

- The use of COMMIT is illustrated in the following simplified sales example, which updates a product's quantity on hand (PROD_QOH) and the customer's balance when the customer buys two units of product 1558-QW1 priced at \$43.99 per unit (for a total of \$87.98) and charges the purchase to the customer's account:

```
UPDATE PRODUCT  
SET PROD_QOH = PROD_QOH - 2  
WHERE PROD_CODE = '1558-QW1';
```

```
UPDATE CUSTOMER  
SET CUST_BALANCE = CUST_BALANCE + 87.98  
WHERE CUST_NUMBER = '10011';
```

```
COMMIT;
```


The Transaction Log

A DBMS uses a **transaction log** to keep track of all transactions that update the database. The DBMS uses the information stored in this log for a recovery requirement triggered by a ROLLBACK statement, a program's abnormal termination, or a system failure such as a network discrepancy or a disk crash. Some RDBMSs use the transaction log to recover a database *forward* to a currently consistent state. After a server failure, for example, Oracle automatically rolls back uncommitted transactions and rolls forward transactions that were committed but not yet written to the physical database. This behavior is required for transactional correctness and is typical of any transactional DBMS.

While the DBMS executes transactions that modify the database, it also automatically updates the transaction log. The transaction log stores the following:

The Transaction Log



- Keeps track of all transactions that update the database. It contains:
 - A record for the beginning of transaction
 - For each transaction component (SQL statement)
 - ▮ Type of operation being performed (update, delete, insert)
 - ▮ Names of objects affected by the transaction (the name of the table)
 - ▮ “Before” and “after” values for updated fields
 - ▮ Pointers to previous and next transaction log entries for the same transaction
 - The ending (COMMIT) of the transaction
- Increases processing overhead but the ability to restore a corrupted database is worth the price

- Increases processing overhead but the ability to restore a corrupted database is worth the price
- If a system failure occurs, the DBMS will examine the log for all uncommitted or incomplete transactions and it will restore the database to a previous state
- The log is itself a database and to maintain its integrity many DBMSs will implement it on several different disks to reduce the risk of system failure

TABLE 9.1 A TRANSACTION LOG

TRL ID	TRX NUM	PREV PTR	NEXT PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				



TRL_ID = Transaction log record ID

PTR = Pointer to a transaction log record ID

TRX_NUM = Transaction number

(Note: The transaction number is automatically assigned by the DBMS.)

CONCURRENCY CONTROL

- Coordinating the simultaneous execution of transactions in a multiuser database system is known as concurrency control.
- The objective of concurrency control is to ensure the serializability of transactions in a multiuser database environment.
- To achieve this goal, most concurrency control techniques are oriented toward preserving the isolation property of concurrently executing transactions.
- Concurrency control is important because the simultaneous execution of transactions over a shared database can create several data integrity and consistency problems.
- The three main problems are lost updates, uncommitted data, and inconsistent retrievals.

- 1. Lost Updates**

- 2. Uncommitted Data**

- 3. Inconsistent Retrievals**

Lost Updates

- The lost update problem occurs when two concurrent transactions, T1 and T2, are updating the same data element and one of the updates is lost (overwritten by the other transaction).
- To illustrate lost updates, examine a simple PRODUCT table. One of the table's attributes is a product's quantity on hand (PROD_QOH).
- Assume that you have a product whose current PROD_QOH value is 35.
- Also assume that two concurrent transactions, T1 and T2, occur and update the
- PROD_QOH value for some item in the PRODUCT table.
- The transactions are shown in Table 10.2.

TABLE 10.2

TWO CONCURRENT TRANSACTIONS TO UPDATE QOH

TRANSACTION	COMPUTATION
T1: Purchase 100 units	$\text{PROD_QOH} = \text{PROD_QOH} + 100$
T2: Sell 30 units	$\text{PROD_QOH} = \text{PROD_QOH} - 30$

TABLE 10.3

SERIAL EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$\text{PROD_QOH} = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH	135
5	T2	$\text{PROD_QOH} = 135 - 30$	
6	T2	Write PROD_QOH	105

Table 10.3 shows the serial execution of the transactions under normal circumstances, yielding the correct answer $\text{PROD_QOH} = 105$.

- suppose that a transaction can read a product's PROD_QOH value from the table before a previous transaction has been committed, using the same product.
- The sequence depicted in Table 10.4 shows how the lost update problem can arise.
- Note that the first transaction (T1) has not yet been committed when the second transaction (T2) is executed.
- Therefore, T2 still operates on the value 35, and its subtraction yields 5 in memory.
- In the meantime, T1 writes the value 135 to disk, which is promptly overwritten by T2. In short, the addition of 100 units is “lost” during the process.

TABLE 10.4

LOST UPDATES

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	$\text{PROD_QOH} = 35 + 100$	
4	T2	$\text{PROD_QOH} = 35 - 30$	
5	T1	Write PROD_QOH (lost update)	135
6	T2	Write PROD_QOH	5

Uncommitted Data

- The phenomenon of uncommitted data occurs when two transactions, T1 and T2, are executed concurrently and the first transaction (T1) is rolled back after the second transaction (T2) has already accessed the uncommitted data—thus violating the isolation property of transactions.
- To illustrate that possibility, use the same transactions described during the lost updates discussion.
- T1 has two atomic parts, one of which is the update of the inventory; the other possible part is the update of the invoice total (not shown).
- T1 is forced to roll back due to an error during the updating of the invoice's total; it rolls back all the way, undoing the inventory update as well.
- This time the T1 transaction is rolled back to eliminate the addition of the 100 units. (See Table 10.5.) Because T2 subtracts 30 from the original 35 units, the correct answer should be 5.

TABLE 10.5

TRANSACTIONS CREATING AN UNCOMMITTED DATA PROBLEM

TRANSACTION	COMPUTATION
T1: Purchase 100 units	$\text{PROD_QOH} = \text{PROD_QOH} + 100$ (Rolled back)
T2: Sell 30 units	$\text{PROD_QOH} = \text{PROD_QOH} - 30$

Table 10.6 shows how the serial execution of these transactions yields the correct answer under normal circumstances.

TABLE 10.6

CORRECT EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$\text{PROD_QOH} = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T1	*****ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	$\text{PROD_QOH} = 35 - 30$	
7	T2	Write PROD_QOH	5

Inconsistent Retrieval

- Inconsistent retrievals occur when a transaction accesses data before and after one or more other transactions finish working with such data.
- For example, an inconsistent retrieval would occur if transaction T1 calculated some summary (aggregate) function over a set of data while another transaction (T2) was updating the same data.
- The problem is that the transaction might read some data before it is changed and other data after it is changed, thereby yielding inconsistent results.

To illustrate the problem, assume the following conditions:

1. T1 calculates the total quantity on hand of the products stored in the PRODUCT table.
2. At the same time, T2 updates the quantity on hand (PROD_QOH) for two of the PRODUCT table's products.

The two transactions are shown in Table

TABLE 10.8	
RETRIEVAL DURING UPDATE	
TRANSACTION T1	TRANSACTION T2
SELECT SUM(PROD_QOH) FROM PRODUCT	UPDATE PRODUCT SET PROD_QOH = PROD_QOH + 10 WHERE PROD_CODE = 1546-QQ2
	UPDATE PRODUCT SET PROD_QOH = PROD_QOH - 10 WHERE PROD_CODE = 1558-QW1
	COMMIT;

- While T1 calculates the total quantity on hand (PROD_QOH) for all items, T2 represents the correction of a typing error: the user added 10 units to product 1558-QW1's PROD_QOH but meant to add the 10 units to product 1546-QQ2's PROD_QOH.
- To correct the problem, the user adds 10 to product 1546-QQ2's PROD_QOH and subtracts 10 from product 1558-QW1's PROD_QOH.
- The initial and final PROD_QOH values are reflected in Table (Only a few PROD_CODE values are shown for the PRODUCT table. To illustrate the point, the sum for the PROD_QOH values is shown for these few products.)

TRANSACTION RESULTS: DATA ENTRY CORRECTION

PROD_CODE	BEFORE PROD_QOH	AFTER PROD_QOH
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	(15 + 10) → 25
1558-QW1	23	(23 - 10) → 13
2232-QTY	8	8
2232-QWE	6	6
Total	92	92

Table 10.7 shows how the uncommitted data problem can arise when the ROLLBACK is completed after T2 has begun its execution.

TABLE 10.7

AN UNCOMMITTED DATA PROBLEM

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$\text{PROD_QOH} = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	$\text{PROD_QOH} = 135 - 30$	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105